

14. Deskewing 2 Channels in the Frequency Domain

© B. Rasnow 28 June 26

Table of Contents

Introduction	1
Simulating simple data	1
arddeskew.m	3
Oscope3.m	5
Adding a deskew GUI	6
Results	6
Comparison of spline and Fourier deskewing at one frequency	7
Comparison of spline and Fourier deskewing at many frequencies	8
Discussion	10
References	11

Introduction

In the previous chapter we addressed the systematic temporal shift (skew) between A0 and A1 in our sampled data. We used MATLAB's `spline` to interpolate and "deskew" A1 to simulate temporal alignment or concurrent sampling with A0. The algorithm implemented in `spline` is somewhat akin to `polyfit`, fitting adjacent points to a cubic (order 3) polynomial and then `polyval` to interpolate the results. The locality of this algorithm works best where the adjacent data is "smooth" with low noise, but it tends to underestimate peaks in high frequency data where the phase changes more between samples. This chapter explores an alternative algorithm for deskewing data in the Frequency Domain. One advantage is this algorithm isn't based on *local* temporal information but instead weights the entire data set. We'll conclude by quantitatively comparing the two algorithms for sine waves of various frequencies from our function generator.

Shifting a sine wave in time is equivalent to adding phase: $\cos(\omega(t + \Delta t)) = \text{Re}(e^{j\omega(t + \Delta t)}) = \text{Re}(e^{j\omega t} e^{j\omega \Delta t})$. So time shifting (by a constant Δt) in the frequency domain is done by multiplying every (complex) frequency component $A(\omega)$ by a phasor $e^{-j\omega \Delta t}$ [1]. Even though each frequency component of the signal must be multiplied by a different phasor, time shifting is mathematically much simpler in the frequency domain! If you doubt this, `>> edit spline` and try to explain what's going on inside that onion.

Simulating simple data

I'm not smart and confident enough to just write down the correct equations and coefficients in MATLAB, so I simulated simple, compact data: a square wave of period 10 samples, and A1 shifted 1 sample relative to A0:

```
npds = 6; % number of periods
a = repmat([zeros(5,1); ones(5,1)],npds,1);
data = [a a([2:end 1])]; % [npds shifted_npds]
plot(data)
```

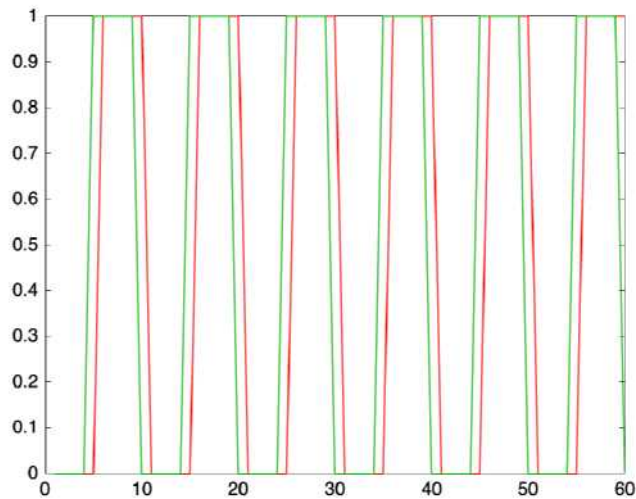


Figure 1. Simulated square waves with a 1 sample shift.

We expect $\Delta t = 1 \text{ sample} * \frac{2\pi \text{ radians}}{10 \text{ samples}}$ and the phasor argument should increase with frequency, i.e., $\Delta\phi = \omega * \Delta t$ (this formula *is* the definition of $\omega \equiv \frac{d\phi}{dt}$ = rate of change of phase). Converting this to a discrete phase shift vector, several trials led to:

```
fd = fft(data);
dph = (0:length(data)-1)' * 2 * pi / 10 / npds;
phasor = exp(-1j * dph);
fd2 = [fd fd(:,2) .* phasor];
rad2deg(angle(fd2(7:12:end,:)))
```

```
ans = 5x3
```

```
108.0000    144.0000    108.0000
144.0000   -108.0000    144.0000
180.0000     0.0000   -180.0000
-144.0000    108.0000   -144.0000
-108.0000   -144.0000   -108.0000
```

The first column are phases of the odd harmonics of simulated A0 square wave, column 2 are phases of the 1-sample shifted square wave A1, and column 3 is phase shifted A1 ideally the same as A0. The first row is the phase of the square wave ([0 0 0 0 1 1 1 1 1 0 0 ...] apparently 90+18 degrees, or $90 + 1/2$ a time sample from a cosine – each time sample = 360 degrees / 10 samples = 36 degrees). $\text{ans}(1,2) = 144$ has 36 degrees more phase resulting from its 1 sample shift, and after phasor multiplication $\text{ans}(1,3)$ it's initial phase was reconstructed.

The 2nd row is the 3rd harmonic, so the 1 sample time shift is $3*36$ degrees added to the initial phase of $\text{ans}(2,1) = 252$ degrees or equivalently (as `fft` computed it), $252-360 = -108$ degrees, and phasor multiplication brought it back to its original $\text{ans}(2,1)=\text{ans}(2,3)$.

The 3rd row is the 5th harmonic, with a phase shift of $5*36 = 180$ degrees (= -180 degrees) ... All phase shifts are as expected.

Let's take the Fourier shifted data (column 3) back to the time domain:

```
fshiftedA1 = abs(ifft(fd2(:,3)));
plot(a - fshiftedA1); ylabel('error')
```

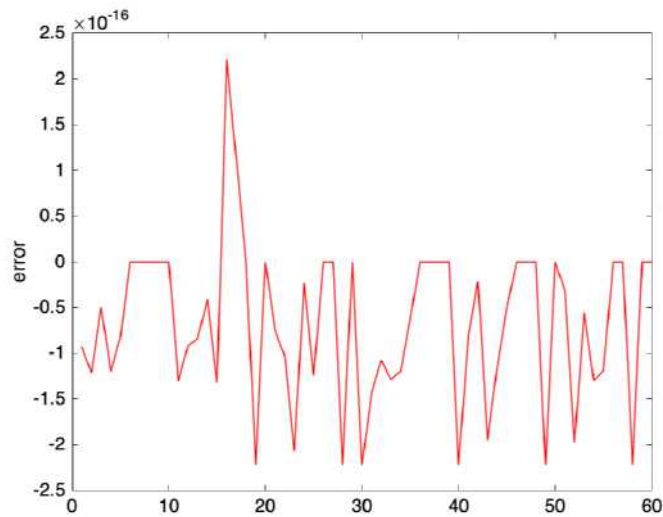


Figure 2. Difference between square wave and Fourier unshifted shifted square wave is $1e-16$, a perfect correction within ~the error of floating point numbers.

arddeskew.m

So let's make a function to deskew in the Fourier domain,

```
function sdata = arddeskew0(data, dphase)
% function sdata = arddeskew(data,dphase)
% deskew arduino data by shifting data(:,2)
% by exp(-j*n*dphase)

if nargin < 2, dphase = 2*pi; end
[r, ~] = size(data);
dph = [0:r-1]' * dphase / (r - 1);
sdata = fft(data(:,2)) .* exp(-j * dph);
sdata = abs(ifft(sdata));
sdata = [data(:,1) sdata];

plot([data(1:50,:) sdata(1:50,2)],'.-');
legend('A0','A1','sA1');
end
```

Question. Show that `arddeskew0(d,2*pi);` works for the square wave, and `arddeskew0(data,2*pi);` overcorrects the skew without distorting the waveform, but `arddeskew0(data,pi);` is a disaster.

Why does one work and not the other took a lot of trial-and-error and pondering over cups of tea. The simulation turned out to be misleadingly easy because the shift was a whole sample. In the case of the Arduino data, its $1/2$ sample, so instead of `dph` ranging from $0 - 2\pi$, with the 2nd half being the negative of the first half by symmetry of $\exp(j*dph)$, changing the range to $0 - \pi$ broke that symmetry and the algorithm failed. Such is the nature of Fourier theory (which you'll encounter in E&M, QM, PDE's, and many engineering domains), here's a solution for Arduino's phase shift of π :

```

function sdata = arddeskew(data)
% function sdata = arddeskew(data)
% deskew arduino data(:,2)
% 18nov20 BR

[r, ~] = size(data);
dph = [1:r/2]' * pi / r; % half of it
dph = [0; dph; -dph(end-1+mod(r,2):-1:1)];
sdata = fft(data(:,2)) .* exp(-j * dph);
sdata = abs(ifft(sdata));
sdata = [data(:,1) sdata];
% for debugging, plot the first 50 points
plot([data(1:50,:) sdata(1:50,2)],'.-');
legend('A0','A1','sA1');
end

```

The last 2 lines helped debugging. I generally separate data processing and visualization for efficient automated processing (without visualization or user input). Combining the two saves a step in the (not trivial) debugging phase. I also deleted the phase argument since it's a constant for the Arduino data. Here's the output on actual data:

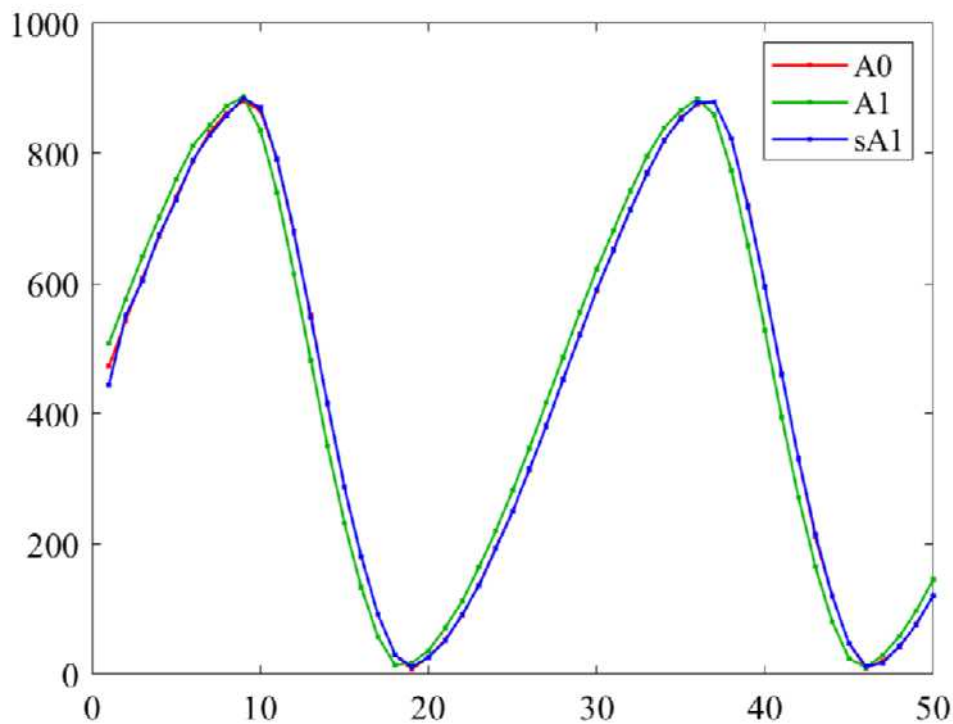


Fig. 3. First 50 samples of Arduino data. The output of `arddeskew()` of A1 mostly is sitting right on top of A0 – it worked.

To explore the performance differences between time and frequency domain deskewing, I rewrote the function to do both. Also added is a tweak to the spline code to fix a frequent error at high frequencies, where the first data point was sometimes excessively negative (I think because there is no data point to its left to balance the spline polynomial).

```

function sdata = arddeskew(data, algorithm)
% function sdata = arddeskew(data, algorithm)
% deskew 2 channel arduino analog data simulating concurrency
% algorithm = 'spline', 'fourier', or 'none' (default)
    if nargin < 2 || lower(algorithm(1)) == 'n' % none
        sdata = data;
        return;
    end
    [r, ~] = size(data);
    if lower(algorithm(1)) == 's' % spline shift
        t = (1:r)';
        timeShiftedA1 = spline(t+.5, data(:,2), t);
        % if the first splined data point is way negative, change it ...
        minVal = min(timeShiftedA1(2:end));
        if timeShiftedA1(1) < minVal, timeShiftedA1(1) = minVal; end
    elseif lower(algorithm(1)) == 'f' % Fourier shift
        dphase = (1:r/2)' * pi / r; % half of it
        dphase = [0; dphase; -dphase(end-1+mod(r,2):-1:1)];
        shiftedA1spectra = fft(data(:,2)) .* exp(-1j * dphase);
        timeShiftedA1 = abs(ifft(shiftedA1spectra));
    end
    sdata = [data(:,1) timeShiftedA1];
end

```

Oscope3.m

Putting these ideas together, here's the nearly final `oscope3.m`. The first block does initialization (once) with 3 `if` statements. The following `while` block tells the microcontroller to measure and send a block of voltages (and elapsed time), processes it, and displays the deskewed (simulated concurrently measured) data in the GUI window.

```

%% oscope3.m -- adds title with freq, amplitudes, phase diff,
% and Fourier deskew
if ~exist('runBtn','var') || ~isvalid(runBtn)
    runBtn = uicontrol('style','radiobutton','string','run', ...
        'units', 'normalized','position',[.13 .93 .1 .04]);
    runBtn.Callback = 'oscope3';
end
if ~exist('ard','var') % initialize arduino
    disp('initializing arduino ...')
    ports = serialportlist;
    ard = serialport(ports{end}, 57600, 'Timeout', 2);
    clear ports;
    readline(ard); % wait for Arduino to reboot
end
if ~exist('runOnce','var'), runOnce = true; end

while runBtn.Value || runOnce
    writeline(ard,'b');
    try
        bin = read(ard,802,'uint16');
        dt = bitshift(bin(802),16)+bin(801); % microseconds
        rawdata = reshape(bin(1:800),2,400)';
    end
end

```

```

% set 2nd arg of trim to ADU value with steepest slope
[rdata, npds] = trim(rawdata, 430);
t = linspace(0,dt/1000,400)';
t = t(1:length(rdata));
sr = length(rawdata)/dt*1e6; % sample rate (#/sec)
freq = sr/(length(rdata)/npds); % Hz, 1/sec

data = arddeskew(rdata, 'fourier');
% data = arddeskew(rdata, 'spline');
fd = fft(data); % data is trimmed to npds
ampl = abs(fd(npds+1,:)) / length(data) * 2;
theta = rad2deg(angle(fd(npds+1,:)));
plot(t, data, '-');
xlabel('msec'); ylabel('ADU'); grid on;
title(sprintf('%.2fHz V1=%.2f V2 = %.2f deg=%.2f', ...
    freq, ampl, diff(theta)), 'FontSize', 12);
displayfps;
catch
    flush(ard);
    fprintf('*');
    break;
end
runOnce = false;
end

```

and manually attach the new callback to the `runBtn` if it was created by `oscope2`:

```
runBtn.Callback = 'oscope3';
```

Adding a deskew GUI

Keeping code simple is important, but simplicity in one place can hide complexity in another. The interface for `arddeskew` above – which algorithm is being called – is buried (hidden) inside `oscope3` code. We can make the algorithm both visible and more easily changeable with a GUI element. Do the following after verifying that `oscope3` is working normally (and maybe save a backup copy just in case ...).

Insert this code inside the `if ~exist('runBtn' ...` block to initialize another control. My reasoning is if `runBtn` doesn't exist, its likely neither does the `skewCtl`, so create it at the same time:

```
skewCtl = uicontrol('Style','popupmenu','Units','normalized',...
    'Position',[.1 .01 .18 .05],'String',{'No deskew','Spline','Fourier'});
```

This adds a popup menu visible at the the bottom of the figure window (Fig. 4), with 3 selectable string choices. But can you think of how to make this GUI element *do* what we want? How to link it to `arddeskew`? And without adding any new lines of code \pause

```
data = arddeskew(rdata, skewCtl.String{skewCtl.Value});
```

Not many programming languages can do this much with so little code!

Exercise: explain how the above line of code works.

Results

Comparison of spline and Fourier deskewing at one frequency

Recall that both A0 and A1 are connected to the same (voltage divided) sine wave from the function generator, so ideally there should be no difference between them (if they were measured concurrently – which they aren't). A difference plot accentuates the errors between A0 and simulated concurrently measured A1, and `rms` converts the difference into a scalar value I denote **err** below. Let's compare the deskewing methods on the last (current) data in the workspace. Scroll up to `oscope3.m` code to refresh your memory ... `rdata = trim(rawdata ...)` is trimmed with the same initial phase and integral number of periods (to within about a sample). First task is to `arddeskew` with Fourier and spline methods, then compute the differences between A0 and the two deskewed A1's, and the original A1 as reference. I'm showing you two ways MATLAB offers to compute the element-wise difference between A0 and A1. Explicitly subtract the columns, or use `diff` on the transposed data matrix (which executes slightly faster on my machine).

```
dataf = arddeskew(rdata,'fourier');  
errf = dataf(:,1)-dataf(:,2);  
errraw = diff(rdata)';  
datat = arddeskew(rdata,'spline');  
errt = diff(datat)';  
plot([errf errt errraw])  
legend('Fourier','Spline','No deskew')  
xlabel('sample'); ylabel('A0-A1')  
title(sprintf('%.1f Hz',freq)); axis tight;
```

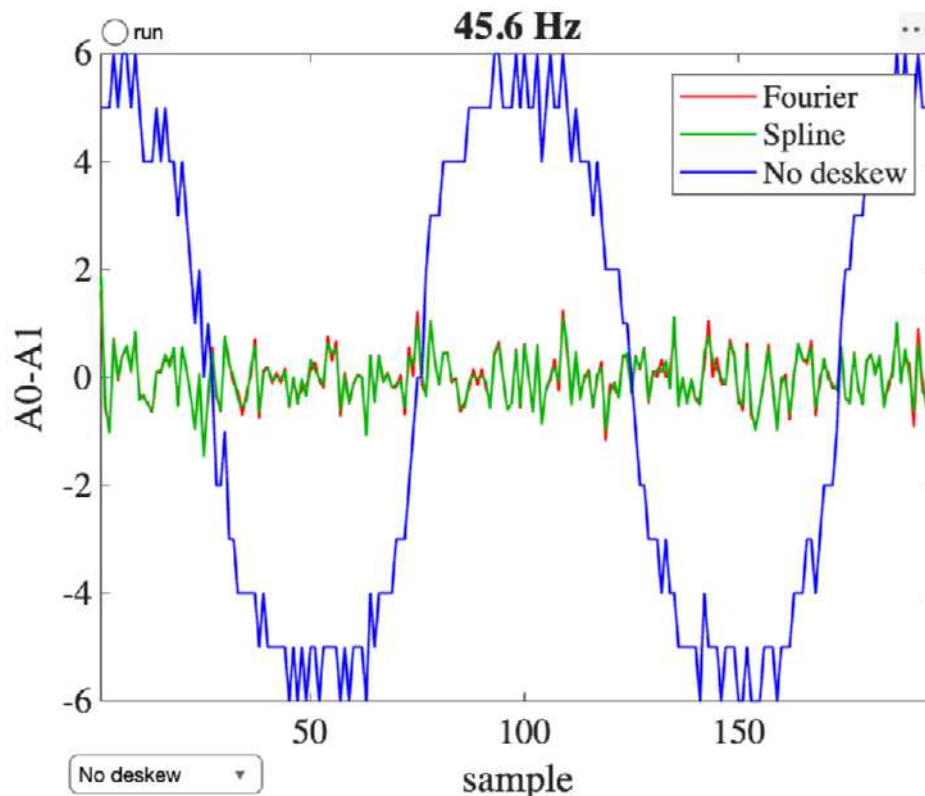


Figure 4. Difference plots. Both deskewing algorithms reduce temporal skew error to <1ADU.

```
rms([errt errf errraw])  
ans =
```

```

    0.4840    0.4898    4.4032
rms([errrt(2:end-1) errrf(2:end-1) errraw(2:end-1)])
ans =
    0.4659    0.4786    4.3966

```

At this low frequency, spline and fourier deskewing reduce a 4.4ADU error from temporal skew to about the same <0.5ADU. A few percent of that error is from single data points at the two ends.

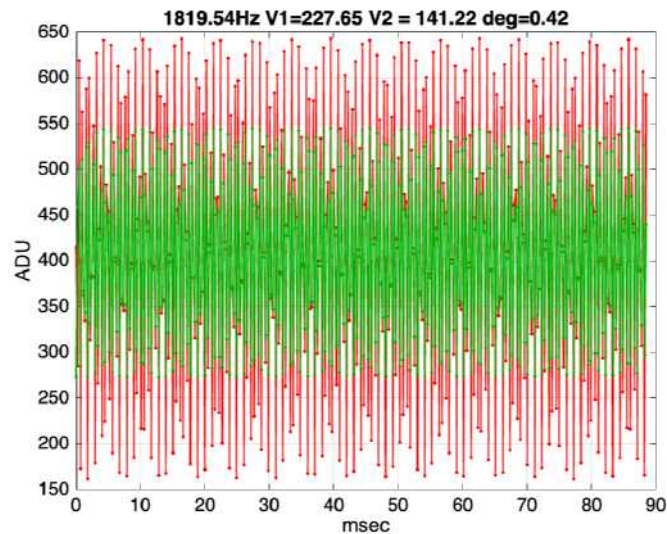
Comparison of spline and Fourier deskewing at many frequencies

If you've implemented Arduino control of the function generator frequency, you could fully automate sweeping by looping over a range of PWM values or frequencies. But I'm going to manually turn the frequency pot while the data is collected "open loop". Start by preallocating storage for data. `oscope3` is configured now to call `arddeskew(rdata,'No deskew')` so to compare with the spline algorithm, we'll have call `arddeskew` again for the same data:

```

numFreqs = 50;
freqs = zeros(numFreqs, 1);
errrt = freqs; % storage for rms(A0-sA1spline)
errrf = freqs; % storage for rms(A0-sA1Fourier)
errn = freqs; % storage for rms(A0-A1) -- no deskew
for i = 1:numFreqs
    pause(.2);
    runOnce = true; oscscope3;
    freqs(i) = freq;
    sdata = arddeskew(rdata,'spline');
    fdata = arddeskew(rdata,'fourier');
    errn(i) = rms(diff(rdata(2:end-1,:))');
    errrt(i) = rms(diff(sdata(2:end-1,:))');
    errrf(i) = rms(diff(fdata(2:end-1,:))');
end

```



```
plot(freqs, [errt errf errn],'.-'); grid;
xlabel('Hz'); ylabel('rms error (ADU)');
legend('Spline','Fourier','no deskew','Location','best')
```

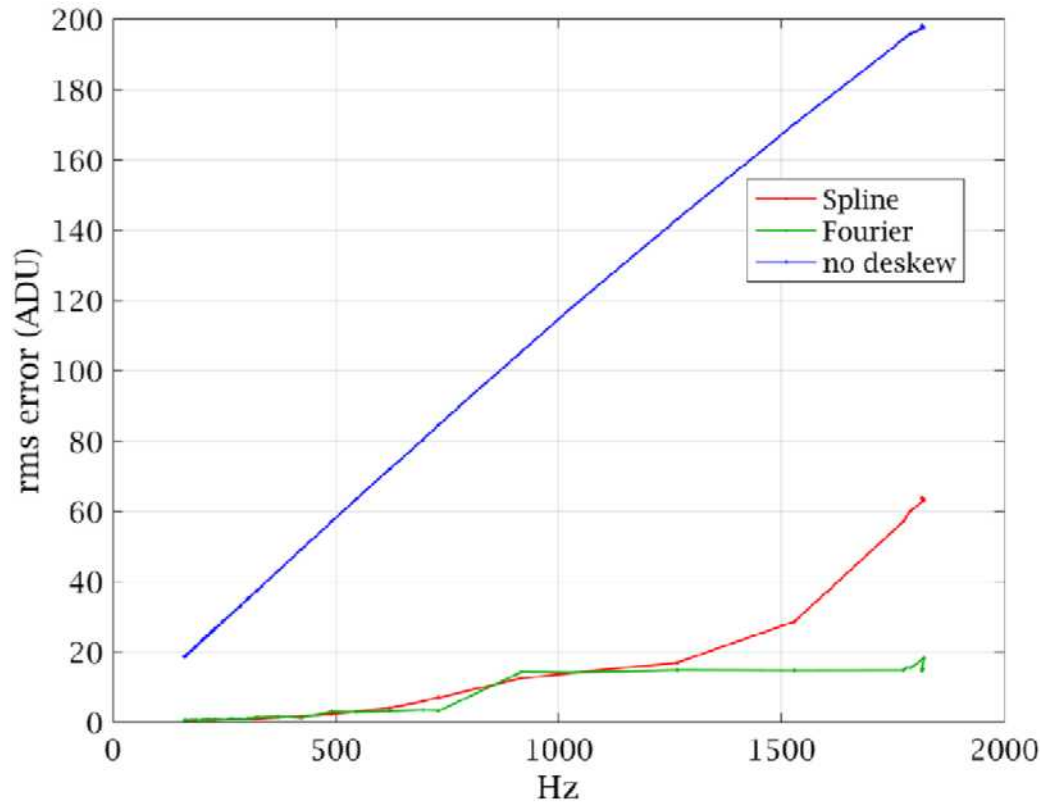


Fig. 5. Skew errors vs. frequency.

Not surprising, no deskewing is a disaster above a few Hz and error increases linearly with frequency. Both deskewing algorithms are remarkably effective (and similar) at low frequencies, but the spline error increases linearly with frequency $> \sim 1500$ Hz, while the frequency domain algorithm doesn't. I truncated the frequencies above half the sample rate

```
sr/2
```

```
ans = 2.2320e+03
```

~ 2.2 kHz = "Nyquist Frequency" [2], where aliasing [3] occurred introducing multiple artifacts. A log plot magnifies the bottom of Fig. 5 to better compare the errors:

```
semilogy(freqs, [errt errf errn],'.-'); grid;
xlabel('Hz'); ylabel('rms error (ADU)');
legend('Spline','Fourier','no deskew','Location','best')
```

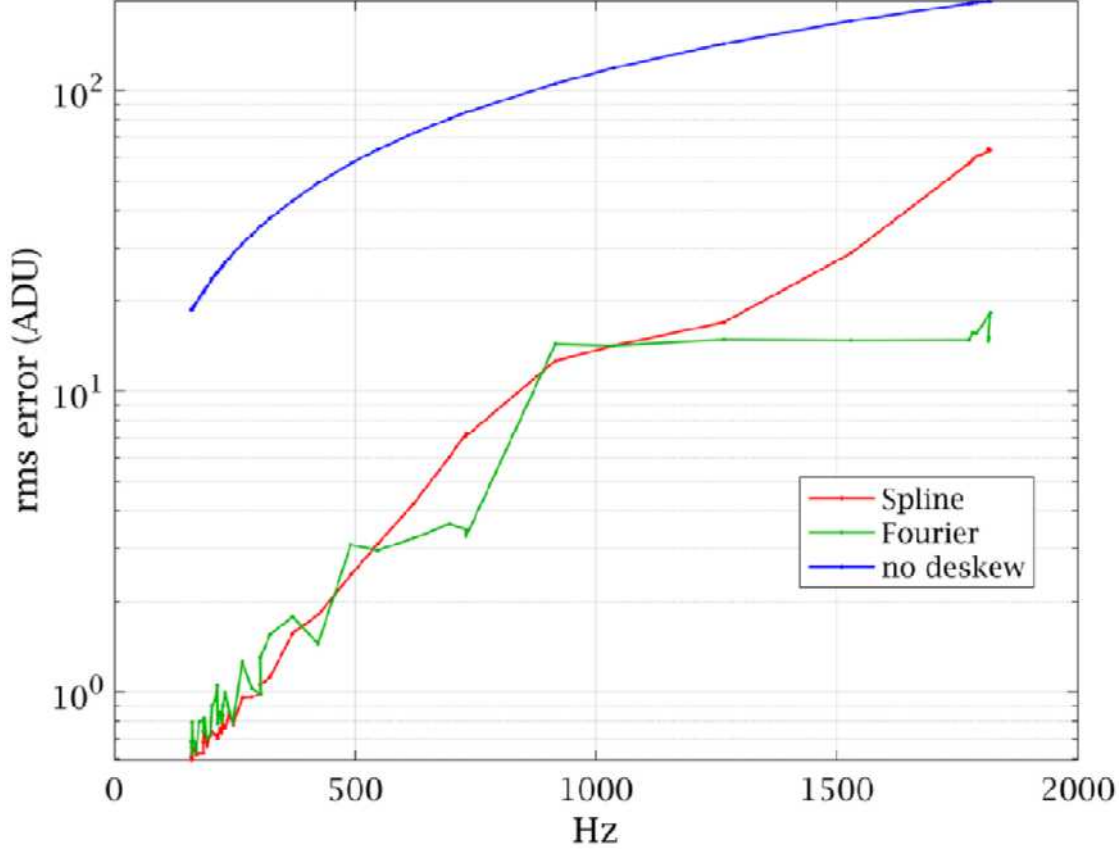


Fig. 6. Skew errors vs. frequency on a semilog plot.

At most of the lowest frequencies, the spline outperforms the frequency domain algorithm. There are probably two factors explaining this: 1) the data is very smooth, and 2) there are few periods of data in the 400 samples for the `fft` to average over. But as the frequency increases both these factors shift in favor of the frequency domain algorithm.

Discussion

The Arduino Nano's analog data acquisition system is remarkably powerful especially considering this device's cost. In previous experiments we showed it can rapidly generate component I-V curve characteristics and transistor $h_{fe} = \beta$ curves with quality indistinguishable from data sheet publications. Here we've applied the Arduino to emulate a 2-channel oscilloscope, and in this dynamic regime, the Arduino is more challenged, because measurements of A0 and A1 aren't simultaneous, but rather are interleaved or multiplexed.

Temporal skew didn't manifest itself earlier because our prior experiments measuring I-V curves and transistor β characteristics involved only low frequencies near DC. Many other tasks such as measuring temperature or other slowly changing parameters won't suffer from temporal skew as long as there is adequate `delay()` or `pause()` following voltage changes, thus letting higher frequency transients decay.

Subtracting (and related differentiation) amplify noise in data. For example, if two similar signals, x and y are measured with 1% uncertainty (i.e., $|\Delta x/x| = |\Delta y/y| = 1\%$), then their difference $d = x - y$ will have uncertainty much higher than 1%. Even if $\langle y \rangle = 0$, the uncertainty of y doesn't vanish and $|\Delta d/d| = \sqrt{(\Delta x/x)^2 + (\Delta y/y)^2} \approx 1.4\%$. But if $\langle y \rangle \approx \langle x \rangle$ then $\langle d \rangle \approx 0$ and the uncertainty explodes.

This argument applies to the *random noise* in the data, which is what's left after removing the *systematic errors* as `arddeskew()` did. Usually the best way to compute a difference between signals is to do the subtraction in hardware with an instrumentation amp [4] or other differential amplifier [5]. One of the key characteristics that predicts the resulting noise is called the "common mode rejection ration" or CMMR [6].

At the start of the semester we characterized 2-terminal devices by computing Thevenin resistances and measuring I-V curves. Here we've taken essentially the same approach a step further in complexity, by characterizing the Arduino Nano as a 2 channel data acquisition system. Empirical testing of ones apparatus is important throughout your scientific and engineering careers. Trust nothing ... if it hasn't been validated. That applies just as well to expensive devices. The reason I would buy an expensive device is to push it to its operational extremes, so validation is never optional. The exercises we've done in the past two units exemplify the kinds of (automated) testing that gives you confidence in your apparatus. Testing and validation often include lots of little tricks. E.g., sticking `dt` at the end of the analog data array, and then reconstructing a 32 bit integer from 2 16 bit ints, or the wonky indexing games to find the right Fourier amplitude and do the phase shifts with the proper symmetry. Those "tricks" get buried in code, covered in layers of abstraction, like an onion, so you don't have to think about them when you work at "higher" levels.

Question. What are the most interesting lessons and questions you got from this activity?

References

- [1] <https://en.wikipedia.org/wiki/Phasor> (Historical note: phasors were introduced to visualize phase relationships in AC circuits in 1894 by C.P. Steinmetz.)
- [2] https://en.wikipedia.org/wiki/Nyquist_frequency
- [3] <https://en.wikipedia.org/wiki/Aliasing>
- [4] https://en.wikipedia.org/wiki/Instrumentation_amplifier
- [5] https://en.wikipedia.org/wiki/Differential_amplifier
- [6] https://en.wikipedia.org/wiki/Common-mode_rejection_ratio